

— ספר GitHub Deployment וספר

לומדת ענף 71

מדריך מלא להעלאה ל-GitHub Deployment Cloud

תוכן עניינים

1. [מבוא](#)
 2. [חלק 1: Git ו-GitHub](#)
 3. [חלק 2: Vercel Deployment](#)
 4. [חלק 3: Railway Deployment](#)
 5. [חלק 4: Database Setup](#)
 6. [חלק 5: CI/CD ו-Auto-Deploy](#)
 7. [חלק 6: Troubleshooting](#)
 8. [חלק 7: Daily Workflow](#)
-

מבוא

למה להעלות ל GitHub?

GitHub = מקום מרכזי לשמור ולשתף קוד.

Your Computer (Local)

↓

GitHub (Remote)

↓

Vercel (Frontend)

Railway (Backend)

↓

World Can Access! 🌐

למה Vercel + Railway?

:Free tier

- Vercel: 0\$ (Frontend)
- Railway: 0\$ עם \$5 (Backend + DB) credit
- **Total: \$0/month!**

חלק 1: Git ו GitHub

1.1 מה זה Git?

Git = Version control system — שומר היסטוריה של כל שינוי בקוד.

Word: "Save" כמו

Word: Save → File → versions

Git: Commit → Repository → full history

למה צריך Git?

✓ **Undo**: אם עשית טעות, אתה יכול לחזור לגרסה קודמת

✓ **Collaboration**: אנשים שונים עובדים על אותו קוד

✓ **Backup**: הקוד שמור בmultiple places

✓ **Tracking**: רואה מי שינה מה ומתי

Git Basics 1.2

:Terminology

Repository (Repo) = תיקיית פרויקט עם היסטוריה

Commit = "save point" בקוד עם הודעה

Branch = גרסה מקבילה של קוד

Push = GitHub-מהמחשב שלך ל commits שליחת

Pull = לחשבון שלך GitHub מ commits הורדת

:Git Workflow

1. Edit files locally

(על המחשב שלך)

2. git add .

(לעקוב אחר קבצים Git-אומר ל)

3. git commit -m "description"

(עם הודעה save point יוצר)

4. git push

(GitHub-ל commits שולח)

1.3 הגדרת Git במachine שלך

:Install Git

```
# Windows: Download from git-scm.com  
# Mac: brew install git  
# Linux: apt-get install git
```

:Configure

```
git config --global user.name "Your Name"  
git config --global user.email "your-email@example.com"
```

GitHub Account 1.4

:Create Account

1. Go to `https://github.com`
2. Sign up with email
3. Verify email
4. You're done!

:GitHub Terms

Profile = Your public page on GitHub

Repository = Folder with code + history

Star = Like button

Fork = Copy someone's repo to your account

Pull Request = Ask to add changes to someone's repo

חלק 2: Vercel Deployment

2.1 מה זה Vercel?

Vercel = Platform להרצת Frontend (React, Vue, Next.js וכו').

```

Your Code (React/Vite)
  ↓
Push to GitHub
  ↓
Vercel watches GitHub
  ↓
Auto-builds & deploys
  ↓
https://your-app.vercel.app (live!)
  
```

יתרונות Vercel:

- Free tier** — unlimited projects ✓
- Auto-deploy** from GitHub ✓
- Fast** — global CDN ✓
- SSL/HTTPS** — free ✓
- Custom domains** — free ✓

:Pricing

```

Free Tier: $0
- Up to 100GB bandwidth/month
- Unlimited projects
- Auto-deploy from Git

Pro: $20/month (if you need more)
  
```

Step-by-Step: Deploy Frontend on Vercel 2.2

Step 1: Create Account

1. Go to `https://vercel.com`
2. Click "Sign up"
3. Choose "GitHub" option
4. Authorize with GitHub account

Step 2: Import Project

1. Dashboard → Add New → Project
2. Click "Import Git Repository"
3. Search for "rf-learning"
4. Select your repo
5. Click "Import"

Step 3: Configure

Vercel will ask

```
Framework Preset: Vite
Root Directory: frontend
Build Command: npm run build
Output Directory: dist
Environment Variables: [see below]
```

Add Environment Variables

```
Key: VITE_API_URL
Value: https://api.railway.app/api
(Update after deploying Backend)
```

Step 4: Deploy

1. Click "Deploy"
2. Wait 30 seconds
3. You get a URL: `https://rf-learning.vercel.app`

Vercel Dashboard 2.3

:Overview

Dashboard shows:

- Deployment history
- Build logs
- Environment variables
- Custom domains
- Analytics

:Important Tabs

Deployments: See all deployments (live, preview, failed)

Settings: Configure build, env vars, git options

Analytics: Traffic, performance

Custom Domain (Optional) 2.4

:Add Custom Domain

1. Vercel Dashboard → Settings → Domains
2. Add your domain
3. Update DNS records (Vercel gives instructions)
4. Wait 24 hours for DNS propagation

:Result

Instead of: `https://rf-learning.vercel.app`
 You get: `https://your-domain.com`

חלק 3: Railway Deployment

3.1 מה זה Railway?

Railway = Platform להרצת Backend (Node.js, Python, Go) וכו'.

```

Your Code (Node.js/Express)
    ↓
Push to GitHub
    ↓
Railway watches GitHub
    ↓
Auto-builds & deploys
    ↓
https://api.railway.app (Backend live!)
    
```

יתרונות Railway:

- Free tier + \$5 credit/month ✓
- Includes PostgreSQL (free!) ✓
- Auto-deploy from GitHub ✓
- Simple dashboard ✓

:Pricing

```

Free: $0 (with $5 credit/month)
    - Enough for: 1 Backend + 1 PostgreSQL

Advanced: Pay-as-you-go
    - $0.09 per hour per instance
    
```

Step-by-Step: Deploy Backend on Railway 3.2

Step 1: Create Account

1. Go to `https://railway.app`
2. Click "Sign up"
3. Choose "GitHub" option
4. Authorize with GitHub

Step 2: Create New Project

1. Dashboard → New Project
2. Select "Deploy from GitHub repo"
3. Select your "rf-learning" repo
4. Confirm

Step 3: Add Services

.Railway auto-detected "backend" folder

:Add PostgreSQL

1. Click "Add Service"
2. Select "Add from Marketplace"
3. Search "PostgreSQL"
4. Click "Create"

Step 4: Configure Environment Variables

:For Backend Service

```
NODE_ENV = production
SERVER_PORT = 3000
JWT_SECRET = your-secret-key-2024
UPLOAD_DIR = /tmp/uploads
```

:For Database Connection (copy from PostgreSQL service)

```
DB_HOST = [copied from PostgreSQL]
DB_PORT = 5432
DB_USER = postgres
DB_PASSWORD = [copied from PostgreSQL]
DB_NAME = rf_learning
```

Step 5: Deploy

1. Railway auto-deploys
2. Check status: "Running" 
3. Get URL from "Domain" tab:
<https://rf-learning-production.up.railway.app>

Railway Dashboard 3.3

:Services

Shows all running services:

- Backend (Node.js)
- PostgreSQL (Database)

:Connect

Click service → Connect tab

Shows connection string for:

- Local development
- Other services

:Deployments

Shows deploy history

Can view logs for each deployment

Can rollback to previous version

Database Connection String 3.4

?What is it

```
postgresql://user:password@host:port/database
```

:Example

```
postgresql://postgres:abc123@railway.example.com:5432/rf_learning
```

:Where to use

Backend: Uses from environment variables

Local dev: Save to .env.local

Tools: psql, pgAdmin, TablePlus

חלק 4: Database Setup

Create Database 4.1

Option A: Railway Console

1. Railway Dashboard → PostgreSQL service
2. Click "Connect" → "Railway CLI"
3. Opens database shell
4. Paste SQL from backend/database/schema.sql
5. Run

Option B: SQL File

```
# Download backup first
pg_dump -U postgres rf_learning > backup.sql

# Then restore if needed
psql -U postgres -d rf_learning < schema.sql
```

Database Schema 4.2

:Tables Created

```

users:
    id, name, id_number, password_hash, role, created_at

courses:
    id, title, description, created_by, created_at

chapters:
    id, course_id, title, description, sort_order, created_at

learning_content:
    id, chapter_id, type, content, file_path, sort_order, created_at

exams:
    id, course_id, title, created_at

questions:
    id, exam_id, text, type, created_at

exam_results:
    id, user_id, exam_id, score, completed_at
  
```

Verify Setup 4.3

:Test Connection

```
# From local machine
psql postgresql://user:pass@host:5432/rf_learning

# You should see prompt:
rf_learning=#
```

:List Tables

```
\dt
```

:Result

```
Should show all tables created
```

Insert Default Data 4.4

:Add Users

```
INSERT INTO users (name, id_number, password_hash, role)
VALUES
  ('Admin User', '999999999', '$2b$10$...', 'admin'),
  ('Test Student', '123456789', '$2b$10$...', 'student');
```

:Add Courses

```
INSERT INTO courses (title, description, created_by)
VALUES
  ('מנועי בוכנה', 'Course about piston engines', 1),
  ('מנועי טילון', 'Course about jet engines', 1);
```

חלק 5: CI/CD ו Auto-Deploy

Auto-Deploy Explained 5.1

:How it works

1. You push code to GitHub:

```
git push origin main
```

2. GitHub notifies Vercel & Railway:

```
"New push detected!"
```

3. Vercel:

- Pulls latest code
- Runs: `npm run build`
- Uploads `dist/` to CDN
- Website updates (5-10 min)

4. Railway:

- Pulls latest code
- Installs dependencies
- Runs: `npm start` (or `dev`)
- Server restarts (2-3 min)

5. Everything is live!

Deploy Process 5.2

:Frontend Deploy Timeline

```
0 sec:    You run: git push
5 sec:    GitHub receives push
10 sec:   Vercel notified
15 sec:   Build starts (npm run build)
30 sec:   Build complete
45 sec:   Upload to CDN
60 sec:   Website updated
```

:Backend Deploy Timeline

```
0 sec:    You run: git push
5 sec:    GitHub receives push
10 sec:   Railway notified
20 sec:   Dependencies installing
30 sec:   Server starting
45 sec:   Health check passes
60 sec:   API live
```


GitHub Actions (Optional) 5.3

?What is it

GitHub Actions = Automated workflows

Run: tests, linting, security checks on every push

:Basic Workflow

```
name: Test & Deploy

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: '18'
      - run: npm install
      - run: npm run build
```

Monitor Deployments 5.4

:Vercel

Dashboard → Deployments

Shows:

- Status (Building, Ready, Error)
- Logs
- Preview URL

:Railway

Dashboard → [Service] → Deployments

Shows:

- Status
- Logs
- Option to rollback

חלק 6: Troubleshooting

Frontend Issues 6.1

"Frontend shows 404 after refresh"

Cause: SPA routing not configured

:Fix in Vercel

```
Settings → Git → Root Directory: frontend
Build: npm run build
Output: dist
```

"Environment variables not loading"

:Solution

1. Add VITE_API_URL in Vercel dashboard
2. Redeploy (click Deployments → Redeploy)
3. Check browser console (F12)

"Slow loading"

:Solutions

1. Check Vercel Analytics
2. Minimize bundle (npm run build)
3. Check network tab in DevTools
4. Enable gzip compression (Vercel auto does this)

Backend Issues 6.2

"Backend returns 500 error"

:Debug

1. Check Railway logs
2. Look for error messages
3. Check environment variables set correctly
4. Restart service (Railway → [Service] → Restart)

"Cannot connect to database"

:Solutions

1. Check DB_HOST, DB_PORT in Railway
2. Verify database is running
3. Check firewall rules
4. Test connection string locally

"File uploads not working"

Cause: UPLOAD_DIR doesn't exist

:Fix

```
Use /tmp/ directory (Railway provides)
UPLOAD_DIR = /tmp/uploads
```

Deployment Issues 6.3

"Vercel deployment stuck"

:Solutions

1. Cancel deployment
2. Check build logs for errors
3. Fix error locally
4. Push again

"Railway keeps crashing"

:Solutions

1. Check logs for errors
2. Verify `NODE_ENV = production`
3. Restart service
4. Check memory usage (might need upgrade)

"Data lost after restart"

Cause: Database not persisted

Fix: Railway PostgreSQL is persistent, but `/tmp/` files are not

Use database for storage, not files

חלק 7: Daily Workflow

Making Changes 7.1

:Edit Code Locally

```
# Make changes to files
# For example: edit backend/src/server.ts

# Test locally
npm run dev
```

:Commit & Push

```
# See what changed
git status

# Stage files
git add .

# Commit with message
git commit -m "Fix login bug"

# Push to GitHub
git push
```

:Auto-Deploy

```
Vercel + Railway auto-deploy
Check dashboards for status
~5-10 minutes to live
```

Version Control Best Practices 7.2

:Commit Messages

✖ Bad:

```
git commit -m "fix"
git commit -m "update"
```

✔ Good:

```
git commit -m "Fix: password validation regex"
git commit -m "Feature: add rich text editor to admin panel"
git commit -m "Refactor: simplify authentication middleware"
```

:Commit Frequency

```
Good: One commit per feature/fix
Not: 100 commits for one feature
Not: One commit with 50 unrelated changes
```

Branching (Optional) 7.3

:Main Branch

```
main = production code
Only working, tested code
Protected branch
```

:Feature Branch

```
feature/login-page = working on new feature
feature/dark-mode = another feature
```

Workflow:

1. `git checkout -b feature/my-feature`
2. Make changes
3. `git commit -m "..."`
4. `git push`
5. Create Pull Request
6. Review & merge to main

Pull Requests 7.4

?What is it

Pull Request = "Request to merge my changes into main"
 Allows: code review, discussion, testing before merge

:Workflow

1. Create feature branch
2. Make changes
3. Push branch
4. Open PR on GitHub
5. Team reviews
6. Merge to main
7. Auto-deploy

Appendix: Full Reference

A. Common Git Commands

```
git init                # Initialize repo
git add .               # Stage all files
git commit -m "msg"    # Create commit
git push               # Send to remote
git pull              # Get from remote
git status            # See changes
git log              # See history
git branch           # See branches
git checkout -b name  # Create branch
git clone <url>       # Download repo
```

B. Environment Variables

:Frontend (.env or Vercel)

```
VITE_API_URL=https://api.railway.app/api
```

:Backend (Railway)

```
NODE_ENV=production
SERVER_PORT=3000
DB_HOST=rail.proxy.rlwy.net
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=...
DB_NAME=rf_learning
JWT_SECRET=your-secret-key
UPLOAD_DIR=/tmp/uploads
```

C. Useful URLs

GitHub: <https://github.com/yairkb100-ai/rf-learning>

Vercel: <https://vercel.com/dashboard>

Railway: <https://railway.app/dashboard>

Frontend: <https://rf-learning.vercel.app>

Backend: <https://rf-learning-production.up.railway.app>

D. Status Codes

:HTTP

```
200 = OK
201 = Created
400 = Bad Request
401 = Unauthorized
404 = Not Found
500 = Server Error
```

:Git

```
✓ = Success
✗ = Failed
⏳ = In Progress
● = Error
```

סיכום

צעדים שעשית:

1.  בנית קוד (Frontend + Backend)
2.  העלית ל-GitHub
3.  כדי להעלות ל-Vercel
4.  כדי להעלות ל-Railway
5.  כדי להעלות Database

מה קיבלת:

Frontend: <https://rf-learning.vercel.app> ✓

Backend: <https://api.railway.app> ✓

Database: PostgreSQL on Railway ✓

Auto-deploy: Every git push = live update ✓

!Cost: \$0/month ✓

כדי להמשיך:

1. עדכן קוד בתוך המחשב
2. `git add . && git commit -m "... " && git push`
3. Vercel + Railway auto-deploy
4. Website updated in 5-10 minutes

Last Updated: 2026-06-28

Version: 1.0

Status: Complete

ספר GitHub ו־Deployment - לומדת ענף 71

עדכון אחרון: 2026-06-28

גרסה: 1.0